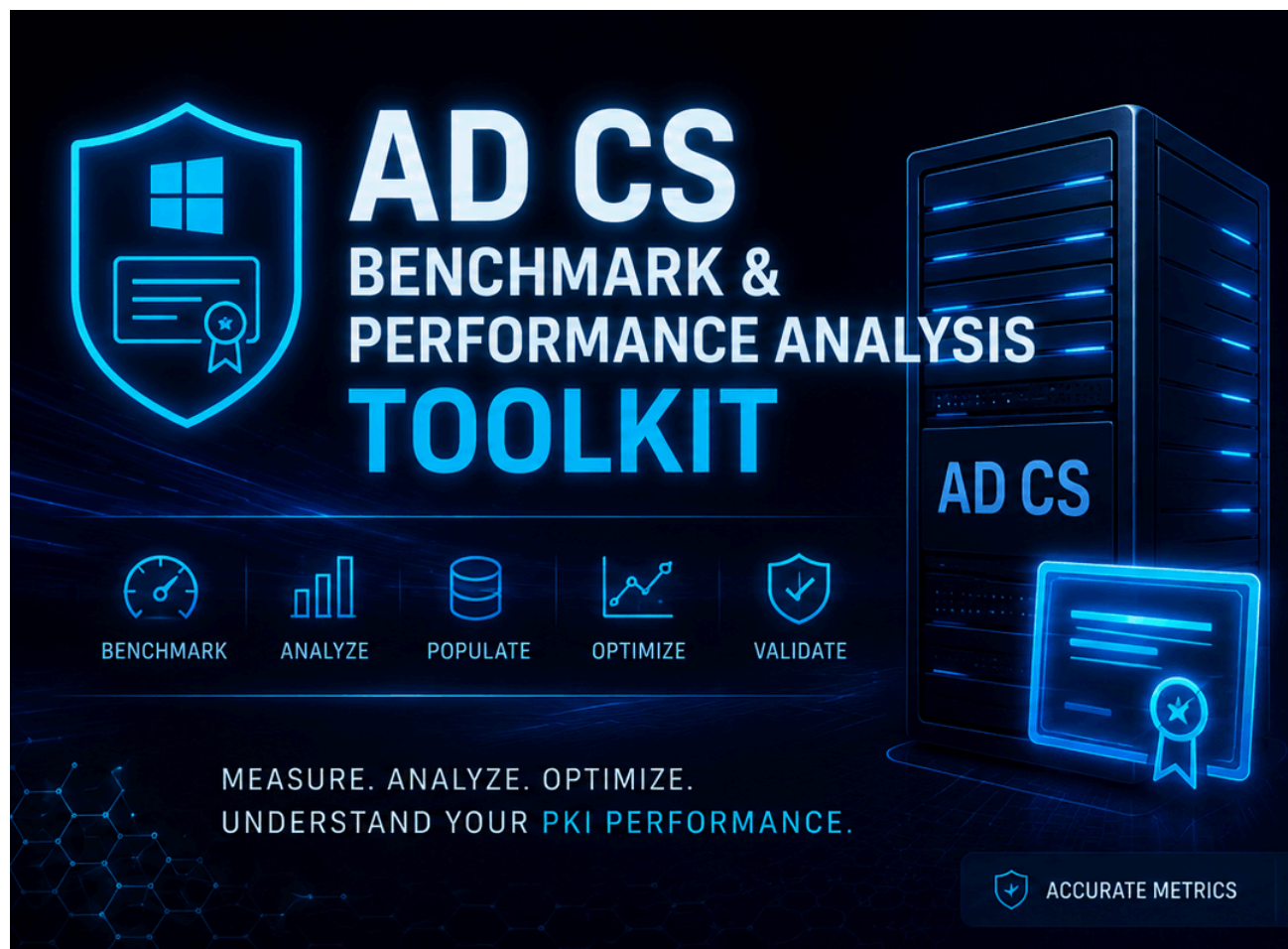


AD CS Performance Toolkit – Michael Waterman

 michaelwaterman.nl/2026/06/15/ad-cs-performance-toolkit

Michael Waterman

June 15, 2026



During many of the Active Directory Certificate Services (AD CS) projects I work on, the discussion always turns to performance. “*So Mike, how to we need to scale the machines?*” Is the virtual hardware powerful enough? Do we have sufficient memory? How many CPU cores should the Certification Authority have? Is our Hardware Security Module (HSM) fast enough? Will disk I/O become a bottleneck? And perhaps the simplest question of all:

How fast is Active Directory Certificate Services actually?

Another challenge I regularly encounter is the need for a realistic Certification Authority database. Whether demonstrating reporting capabilities, validating certificate templates, testing custom PowerShell scripts, or creating a representative lab environment, an empty CA database simply does not reflect production. After searching for a suitable solution, I realized that nothing really existed that combined realistic certificate generation with meaningful performance benchmarking. Existing tools either focused on enrollment or

stress testing, but none provided detailed insight into where certificate issuance time was actually spent....so I built one.

The *AD CS Benchmark & Performance Analysis Toolkit* is a PowerShell-based utility that generates realistic PKCS#10 certificate requests, submits them through the native ICertRequest2 COM interface, optionally creates matching Active Directory objects, and records detailed timing information for every stage of the enrollment process.

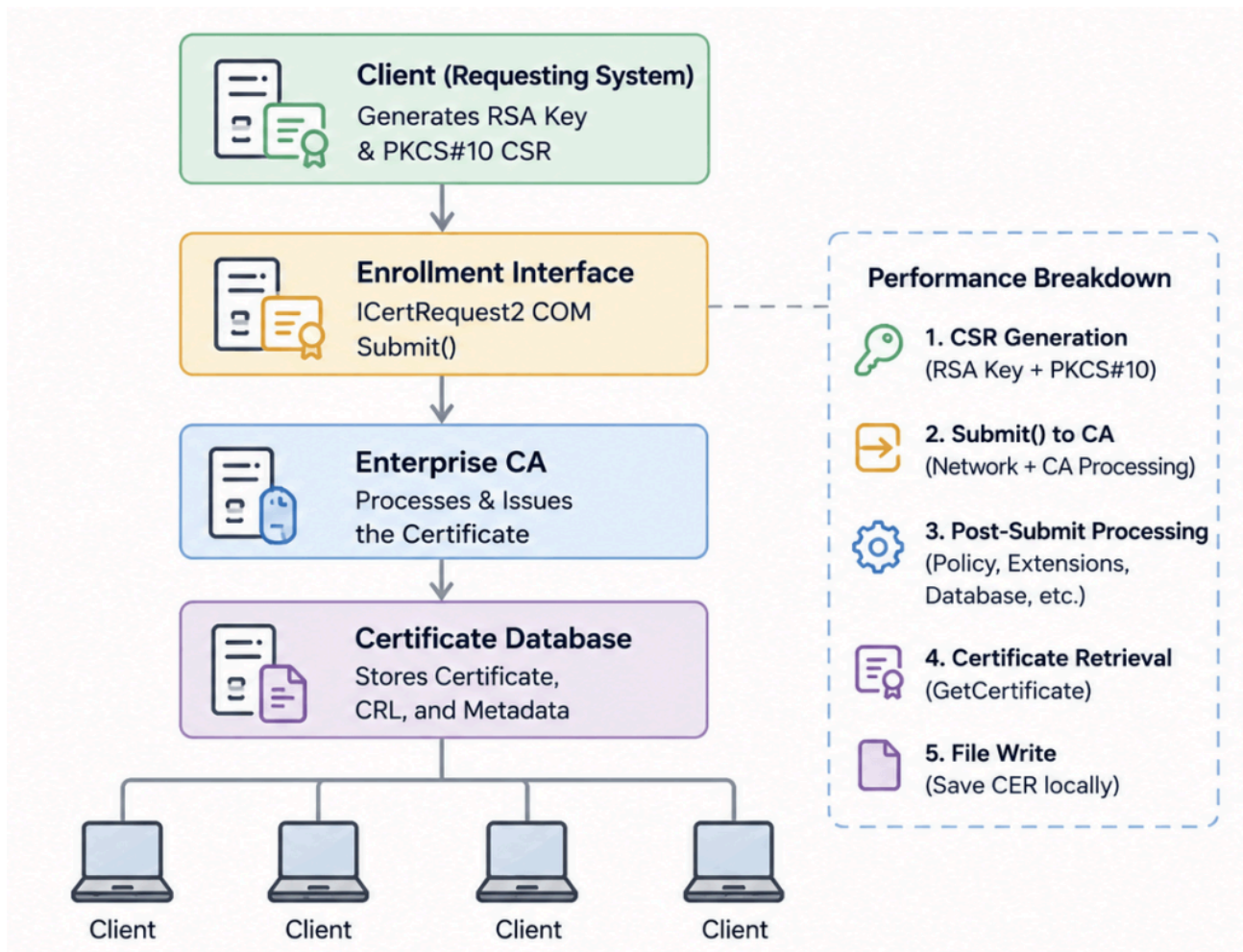
The tool can populate a Certification Authority databases with realistic data, validate certificate templates, benchmark hardware platforms, evaluate policy modules, compare cryptographic configurations, and generate representative datasets for demonstrations and reporting.

Like many others, I initially assumed that increasing RSA key sizes would primarily increase the workload on the Certification Authority. The measurements told a completely different story. Even with very large key sizes, the CA itself continued to process requests within only a few milliseconds. Almost the entire execution time was spent on the requesting system while generating the private key and PKCS#10 request. That insight alone fundamentally changed how I look at AD CS performance, and it became the primary motivation for sharing this toolkit with the wider PKI community.

Let's dive in!

Why measuring AD CS performance is hard

At first glance, benchmarking Active Directory Certificate Services appears to be straightforward. Generate a certificate request, submit it to the Certification Authority, measure the elapsed time, and repeat the process a few thousand times, yeah right...In reality, the measured execution time consists of several independent operations. Before the Certification Authority even receives a request, the client must generate a private key, create a PKCS#10 certificate signing request, write temporary files, and invoke the enrollment interface. Only then does the CA process and issue the certificate.



The certificate request process

As a result, a simple stopwatch around `certreq.exe` tells you very little about the actual performance of the Certification Authority itself.

This distinction becomes increasingly important when comparing hardware platforms, evaluating Hardware Security Modules (HSMs), or validating the impact of larger RSA key lengths. Without separating client-side cryptographic operations from server-side processing, it is easy to draw incorrect conclusions about where the performance bottleneck actually resides. Another challenge is the lack of realistic test data. Many AD CS laboratories contain only a handful of certificates, making it difficult to test reporting scripts, certificate inventory tools, policy modules, or management procedures under conditions that resemble production.

These observations led to a simple design goal: *create a toolkit that not only generates realistic certificate requests but also measures every stage of the enrollment process independently, allowing PKI administrators to understand exactly where time is spent.*

Building the “AD CS Benchmark & Performance Analysis Toolkit”

Fancy name, right! The toolkit was designed around a simple principle, benchmark the Certification Authority without introducing unnecessary overhead. Rather than relying on

external load-testing frameworks, it uses native Windows components throughout the entire enrollment process. Certificate requests are generated using `certreq.exe -new`, after which they are submitted directly through the ICertRequest2 COM interface used by Active Directory Certificate Services itself. This provides a realistic representation of how enterprise certificate enrollment operates.

For environments that require more representative testing, the toolkit can optionally create matching Active Directory user or computer objects before enrollment. This allows certificate templates that depend on directory attributes to behave exactly as they would in production while simultaneously populating the CA database with realistic records. Several design decisions were made to improve the quality and reproducibility of benchmark results.

Warm-up requests can be executed before measurements begin, preventing one-time initialization tasks from influencing the final statistics. A configurable random seed ensures that repeated benchmark runs generate the same template distribution, making it possible to compare hardware platforms or configuration changes under identical conditions. Perhaps the most valuable feature is the separation of timing measurements. Instead of reporting only a single execution time, the toolkit records the duration of CSR generation, certificate submission, post-processing activities, optional certificate retrieval, and the total operation. This makes it immediately apparent whether delays originate from client-side cryptographic operations or from the Certification Authority itself.

Although originally developed as a benchmarking utility, the project evolved into a multifunctional tool. It can rapidly populate Certification Authority databases, validate certificate templates, test policy and exit modules, generate representative datasets for reporting, and provide objective performance data for hardware sizing and capacity planning.

Key features

The AD CS Benchmark & Performance Analysis Toolkit was designed to provide realistic certificate enrollment scenarios while collecting meaningful performance data. Rather than acting as a simple certificate generator, it combines benchmarking, automation, and CA database population into a single PowerShell utility.

Some of its key capabilities include:

- *Generate thousands of realistic certificate requests* using native Windows enrollment components.
- *Support multiple certificate templates* to simulate mixed enterprise environments.
- *Measure individual enrollment stages*, including CSR generation, CA submission, certificate retrieval, and total execution time.

- Execute *warm-up runs* that are excluded from benchmark statistics to improve measurement accuracy.
- Produce *deterministic benchmark* results through a configurable random seed, enabling reproducible comparisons between hardware platforms and configuration changes.
- Optionally create matching *Active Directory objects* for templates that depend on directory information.
- Automatically *clean up* test objects after execution, allowing repeated testing without manual intervention.
- *Export detailed metrics to CSV* for further analysis or visualization.
- *Log failed or denied requests* separately to simplify troubleshooting.
- *Populate realistic Certification Authority databases* for reporting demonstrations, inventory scripts, and management tooling.

One of the primary design goals was to separate client-side cryptographic operations from server-side processing. By measuring each stage independently, the toolkit provides a much clearer understanding of certificate enrollment performance than a simple end-to-end stopwatch measurement. Although originally developed for internal benchmarking purposes, the toolkit has evolved into a practical utility for PKI administrators, consultants, and architects who require objective performance data and realistic laboratory environments.

Using the toolkit

The complete source code for the AD CS Benchmark & Performance Analysis Toolkit is available on my GitHub repository, where it can be downloaded, modified, and extended for your own lab or benchmarking requirements.

GitHub repository: [mfgjwaterman/AD-CS-Performance-Toolkit](https://github.com/mfgjwaterman/AD-CS-Performance-Toolkit): [Hosts the repository of the AD CS Performance Toolkit](#)

The toolkit consists of a single PowerShell script and was designed to be self-contained with minimal external dependencies. It leverages native Windows components such as `certreq.exe` for PKCS#10 request generation and the ICertRequest2 COM interface for certificate enrollment, making the benchmark process closely resemble real-world enterprise certificate issuance. While the script offers a wide range of parameters, most users will only need a few to get started.



The execution flow

Basic benchmark

The simplest benchmark generates a number of certificate requests and displays a performance summary at the end.

```
.\Invoke-AdcsBenchmark.ps1 `
    -Count 1000 `
    -Performance
```

This provides a quick indication of overall enrollment performance and is useful for validating new Certification Authority deployments or comparing hardware platforms.

Repeatable benchmarks

When comparing different environments, reproducibility is essential. By specifying a random seed, the toolkit generates the same template distribution for every benchmark run, ensuring that results remain comparable.

```
.\Invoke-AdcsBenchmark.ps1 `
    -Count 5000 `
    -WarmupCount 25 `
    -Seed 42 `
    -Performance
```

Combined with the warm-up phase, this eliminates much of the variation caused by initialization overhead and random template selection.

Simulating a production environment

Real enterprise environments rarely issue a single certificate type. The toolkit therefore supports multiple templates within a single benchmark run, providing a more representative workload.

```
.\Invoke-AdcsBenchmark.ps1 `
    -Count 5000 `
    -Templates CorpComputer,CorpWebServer,CorpUser `
    -Performance
```

This allows administrators to evaluate the impact of different certificate templates and enrollment policies under realistic conditions.

Note! More examples are listed on my Github page.

Creating realistic data

One feature that proved useful during development was the ability to populate a Certification Authority database with realistic data. By optionally creating matching Active Directory users or computer objects before enrollment, the resulting database closely resembles a production environment.

This is especially valuable when developing reporting tools, inventory scripts, certificate lifecycle automation, or demonstrating PKI management capabilities. Instead of working with a nearly empty CA database, administrators can generate thousands of realistic certificate records within a relatively short period of time. Ultimately, the toolkit is not only intended to benchmark performance, but also to simplify the creation of realistic PKI laboratory environments for testing, development, and demonstrations.

Test environment

Let’s put the script to the test, see what happens. *The benchmark results presented below were obtained in a controlled lab environment and should be interpreted as reference values rather than absolute performance figures. Actual results will vary depending on hardware, storage performance, virtualization platform, certificate templates, cryptographic providers, and Active Directory integration.* The Certification Authority was configured as follows:

Component	Specification
Operating System	Windows Server 2025
CA Type	Enterprise Certification Authority
Memory	8 GB DDR4
Virtual Disk	127 GB
Storage	NVMe-backed storage
Virtual CPUs	8 vCPUs
Host Processor	AMD Ryzen 7 5825U (8 cores / 16 threads)

The benchmark was executed using RSA 2048-bit keys with a warm-up phase to eliminate initialization overhead before measurements were recorded. Although this configuration represents a relatively modest virtual machine, it demonstrates that the Certification Authority itself contributes only a small fraction of the total enrollment time. In the measured scenarios, the majority of the execution time was spent generating the RSA key pair and PKCS#10 request on the requesting system rather than processing the request on the Certification Authority. The purpose of these benchmarks is therefore not to establish universal performance figures, but to illustrate how the toolkit can be used to identify bottlenecks and objectively compare different hardware platforms, template configurations, and cryptographic settings.

Example benchmark run

The benchmark run used three certificate templates that represent the most common enrollment scenarios: *Computer*, *User*, and *Web Server* certificates. Rather than using the default Microsoft templates, these templates were modified specifically for benchmarking purposes. Enrollment requirements such as manager approval and unnecessary dependencies were removed, while preserving realistic behavior and certificate contents. This allows the benchmark to focus on certificate enrollment performance without

administrative workflows influencing the results. Together, these templates exercise different code paths within Active Directory Certificate Services and provide a representative workload for performance testing. The following example executes a benchmark using the *TemplateBenchmark* mode.

```
.\Invoke-AdcsBenchmark.ps1 `
  -Count 100 `
  -CAConfig "DEMO-SRV-01.corp.com\CORP-ROOT-CA" `
  -Templates CorpComputer,CorpUser,CorpWebServer `
  -WarmupCount 5 `
  -KeyLength 2048 `
  -Performance `
  -TemplateBenchmark
```

Unlike a traditional mixed benchmark, the *TemplateBenchmark* parameter instructs the toolkit to execute an independent benchmark for each configured certificate template. Instead of combining all requests into a single result set, every template is processed separately using identical benchmark settings.

This approach makes it possible to compare enrollment performance across different certificate types and immediately identify templates that require additional processing due to policy evaluation, directory lookups, publication settings, or custom extensions.

Measured Benchmark Summary									
Template	Requests	Succeeded	Failed	AverageCsrMs	AverageSubmitMs	MedianSubmitMs	P95SubmitMs	P99SubmitMs	StdDevSubmitMs
CorpUser	100	100	0	477,43	23,83	23,08	28,22	33,75	2,41
CorpComputer	100	100	0	361,27	24,65	24,18	27,4	33,55	2,36
CorpWebServer	100	100	0	368,67	23,72	23,19	27,51	28,92	1,73

AD CS Performance Benchmark Report									
CA	: DEMO-SRV-01.corp.com\CORP-ROOT-CA								
Mode	: TemplateBenchmark								
Templates	: CorpUser, CorpComputer, CorpWebServer								
Warm-up Requests	: 5								
Measured Requests	: 300								
Succeeded	: 300								
Failed	: 0								
Total Runtime Seconds	: 136.64								
Overall Rating	: Excellent								

Latency - CA Submit Only									
Average CA Submit Only	: 24.07 ms								
Median CA Submit Only	: 23.49 ms								
P95 CA Submit Only	: 27.62 ms								
P99 CA Submit Only	: 33.55 ms								
Fastest Submit	: 21.75 ms								
Slowest Submit	: 43.1 ms								
StdDev Submit	: 2.23 ms								

Breakdown									
Average CSR Generation	: 402.46 ms								
Average CA Submit Only	: 24.07 ms								
Average Post Submit	: 0.02 ms								
Average Cert Retrieval	: 0 ms								
Average Cert File Write	: 0 ms								
Average Total Operation	: 431.67 ms								

Share of Total Operation									
CSR Generation Share	: 93.2333 %								
CA Submit Only Share	: 5.576 %								
Post Submit Share	: 0.0046 %								
Cert Retrieval Share	: 0 %								
Cert File Write Share	: 0 %								

Throughput									
Certificates / Second	: 2.2								
Certificates / Minute	: 131.73								
Certificates / Hour	: 7903.8								

Template Summary									
Template	Requests	Succeeded	Failed	AverageSubmitMs	MedianSubmitMs	P95SubmitMs	P99SubmitMs	StdDevSubmitMs	AverageTotalMs
CorpUser	100	100	0	23,83	23,08	28,22	33,75	2,41	506,27
CorpComputer	100	100	0	24,65	24,18	27,4	33,55	2,36	391,03
CorpWebServer	100	100	0	23,72	23,19	27,51	28,92	1,73	397,72

Benchmark results

After the test completes, the toolkit automatically generates a detailed performance report containing both a high-level summary and an in-depth latency analysis. The *Measured Benchmark Summary* provides a concise overview of every template, including the number of successful requests and key latency statistics such as the average, median, P95, and P99 submission times. These values provide insight into both average performance and consistency under load. The AD CS Performance Benchmark Report then breaks down the complete enrollment operation into its individual stages:

- *CSR Generation* – generation of the RSA key pair and PKCS#10 certificate request.
- *CA Submit Only* – the actual processing and issuance performed by Active Directory Certificate Services.
- *Post Submit* – additional processing after certificate issuance.
- *Certificate Retrieval* – optional retrieval of the issued certificate from the Certification Authority.
- *Certificate File Write* – optional storage of the retrieved certificate on disk.

One of the most useful sections of the report is *Share of Total Operation*, which expresses each phase as a percentage of the complete enrollment process. In my own benchmark environment, the results clearly demonstrated that *CSR generation accounts for well over 90% of the total execution time*, while the Certification Authority itself typically requires only a few tens of milliseconds to evaluate and issue the certificate. This finding fundamentally changes the way certificate enrollment performance should be interpreted. When overall enrollment appears slow, the bottleneck is often not the Certification Authority at all, but the requesting system performing the cryptographic operations.

The report also includes throughput metrics such as *certificates per second, per minute, and per hour*, making it easy to compare different hardware platforms or validate infrastructure changes using objective measurements.

Finally, the *Template Summary* consolidates all benchmark results into a compact overview. Because every template is benchmarked independently, differences immediately become visible and can help identify additional processing introduced by template configuration, policy modules, directory integration, or publication settings. SO at the end of the day you can now easily answer the question, *“Which part of the certificate enrollment process is actually responsible for the observed performance and what matters to me?”*

Understanding the benchmark results

Running a benchmark is only the first step. The real value lies in understanding what the collected metrics actually represent. Unlike traditional benchmarking tools that report only a single execution time, the AD CS Benchmark & Performance Analysis Toolkit measures multiple stages of the certificate enrollment process independently. This makes it possible to identify where time is actually being spent. Let's use a snippet from the results above to explain.

Template	Requests	AverageCsrMs	Average SubmitMs	AverageTotalMs
CorpComputer	100	361,27	24,65	391,03
CorpWebServer	100	368,67	23,72	397,72
CorpUser	100	477,43	23,83	506,27

At first glance, many people focus on the *AverageTotalMs* column. However, this value represents the complete end-to-end enrollment operation and includes client-side cryptographic operations as well as server-side processing.

The *AverageCSRMs* metric measures the time required to generate the RSA key pair and create the PKCS#10 certificate request. Depending on key length and hardware capabilities, this often represents the largest portion of the total execution time.

The *AverageSubmitMs* metric measures the actual interaction with the Certification Authority. This includes transmitting the request, evaluating certificate policies, processing the request, and issuing the certificate.

Finally, *AverageTotalMs* combines all measured stages into a single end-to-end execution time.

Note! The reported “*AverageTotalMs*” represents the complete end-to-end enrollment time as observed by the toolkit. It includes CSR generation, certificate submission, post-processing, optional certificate retrieval and file operations, together with a small amount of PowerShell execution overhead. As a result, the sum of the individual measured phases may not equal exactly 100% of the total operation.

One of the most interesting observations during testing was that the Certification Authority itself contributed only a very small percentage of the total runtime. In most benchmark scenarios, more than 90 percent of the execution time was spent on the requesting system while generating the RSA key pair and PKCS#10 request. This has important implications for capacity planning. When benchmark results appear slow, upgrading the Certification Authority may have little impact if the bottleneck actually resides on the client performing the cryptographic operations.

The toolkit also provides additional statistics such as median latency, 95th percentile, 99th percentile, and standard deviation. These values offer valuable insight into performance consistency and can reveal intermittent delays that are not visible when considering averages alone. Rather than simply answering the question “*How fast is my CA?*”, the benchmark provides a much more useful answer: “*Where is time actually being spent during certificate enrollment?*”

Future development

Although the toolkit already provides a comprehensive framework for benchmarking Active Directory Certificate Services, several enhancements are planned for future releases.

One of the primary goals is to further reduce the dependency on native Windows utilities by moving more functionality into native PowerShell. While `certreq.exe` remains a reliable and well-supported component for generating PKCS#10 requests, a more PowerShell-centric implementation would provide greater flexibility and simplify integration into automated testing and CI/CD pipelines. Another area of interest is support for additional cryptographic algorithms. The current implementation focuses on RSA-based certificates, which remain the most widely deployed in enterprise environments. However, future versions may include support for Elliptic Curve Cryptography (ECC) and post-quantum cryptographic algorithms, allowing admins to benchmark and compare the performance of both classical and next-generation cryptographic primitives within a single framework.

Reporting capabilities are another area that can be expanded. While the current console output and CSV exports provide detailed performance information, future versions may include graphical reports, HTML summaries, and trend analysis to simplify comparison between benchmark runs. But that's yet to be seen.

Another feature I would like to explore is support for submitting requests under a different requester identity. At the moment, I want to avoid relying on tools such as PsExec to run the process as another user. A more native PowerShell-based approach would make it possible to test requester-specific enrollment behavior, permissions, template access, and audit logging in a cleaner and more controlled way.

Finally, the toolkit has the potential to evolve beyond performance testing alone. Features such as automated comparison between Certification Authorities, Cryptographic Service Providers, Hardware Security Modules, and different template configurations could make it a valuable validation and capacity planning tool for enterprise PKI deployments.

Final word

What started as a simple attempt to fill a CA database with certificate data turned into measuring the performance of Active Directory Certificate Services, quite the adventure I must say. Along the way, the toolkit proved to be useful not only for benchmarking, but also for populating realistic Certification Authority databases, validating certificate templates, testing policy modules, comparing hardware platforms, and creating representative laboratory environments for demonstrations and development.

This reinforces the importance of measuring rather than assuming. Performance bottlenecks are not always located where we expect them to be, and objective benchmark data provides a much stronger foundation for hardware sizing, architectural decisions, and troubleshooting than theoretical calculations alone.

I hope this toolkit proves useful to others who need realistic environments or want to better understand the performance characteristics of their own Active Directory Certificate Services deployment.

Key takeaway

During development of this toolkit, one observation stood out above all others, the Certification Authority itself was rarely the bottleneck. Even with large RSA key sizes, the majority of the enrollment time was spent generating the private key and PKCS#10 request on the client system. Measuring each phase independently fundamentally changed the way I interpret certificate enrollment performance.

As the project continues to evolve, feedback, ideas, and contributions from the community are always welcome. For now there's version 1.0, more to come.

Until next time!